C 语言训练营专业阶段 20 题（最终版）

1. 单链表实现约瑟夫环问题

题目：用单链表模拟约瑟夫环问题。给定总人数 n、起始报数位置 k（从 1 开始计数）、报数阈值 m（数到 m 的人出列），按出列顺序输出所有人的编号。

要求：

- 链表节点包含 id（编号）和 next 指针。
- 输入：n=5, k=1, m=2（5 人围成环，从 1 号开始数，数到 2 出列）。
- 输出：2 → 4 → 1 → 5 → 3（示例）。

测试输入输出：

输入：n=5, k=1, m=2 → 输出：2 4 1 5 3（用空格分隔）。

考察点：链表遍历、循环指针操作、边界条件（如 m=1 时直接全部出列）。

2. 双向循环链表实现队列

题目：用双向循环链表实现一个队列（FIFO），支持入队（enqueue）和出队（dequeue）操作。

要求：

- 链表节点包含 data（存储整数）和 prev/next 指针。
- 队列初始化为空，入队顺序为 1→2→3，出队顺序应为 1→2→3。
- 输入：依次调用 enqueue(1)、enqueue(2)、enqueue(3)，然后调用 dequeue() 三次。
- 输出：1 2 3（每次 dequeue 的结果）。

测试输入输出：

入队顺序 [1,2,3] → 出队输出：1 2 3。

考察点：双向链表的插入/删除、循环链表的边界处理（头尾相连）。

3. 环形链表实现约瑟夫环问题

题目：用环形链表（非循环链表改造）模拟约瑟夫环问题，要求利用环形结构的特性简

化节点连接操作。

要求：

- 环形链表通过 next 指针首尾相连，无需额外头节点。
- 输入：n=5, k=1, m=2（同题目 1）。
- 输出：与题目 1 一致（2 4 1 5 3）。

测试输入输出：同题目 1。

考察点：环形链表的构建（尾节点指向头节点）、约瑟夫环的循环淘汰逻辑。

4. 二叉树的前序遍历

题目：实现二叉树的前序遍历（根→左→右），输出遍历结果。

要求：

- 二叉树节点包含 val（整数值）、left 和 right 指针。
- 输入：通过层次遍历数组构建二叉树（如 [1,2,3,null,null,4,5] 表示根 1，左 2，右 3，3 的左 4，右 5）。
- 输出：前序遍历结果 1 2 3 4 5（示例）。

测试输入输出：

输入层次数组 [1,2,3,null,null,4,5] → 输出：1 2 3 4 5。

考察点：递归/非递归遍历实现、二叉树的构造（处理 null 节点）。

5. 排序二叉树统计字母频率

题目：使用排序二叉树（BST）统计文本文件中英文字母（不区分大小写，仅统计 a-z）的出现次数，按字母顺序输出结果。

要求：

- BST 节点包含 char c（字母）、count int（次数）、left/right 指针。
- 输入文本文件 test.txt 内容示例："AbcAb"（转换为小写后为 abcab）。
- 输出：按字母顺序打印 a:2 b:2 c:1。

测试输入输出：

输入文件内容 AbcAb → 输出：a:2 b:2 c:1。

考察点：BST 的插入与遍历（中序遍历输出有序序列）、文件读取与字符处理（忽略非字母）。

6. Hash 表统计单词频率并排序

题目：使用 Hash 表统计文本文件中单词（以空格/标点分隔，转换为小写）的出现次数，按次数降序排序，次数相同则按字母升序排序。

要求：

- Hash 表使用链地址法解决冲突，哈希函数可选 djb2。
- 输入文本文件 test.txt 内容示例："hello world hello"。
- 输出：hello:2 world:1。

测试输入输出：

输入文件内容 hello world hello → 输出：hello:2 world:1。

考察点：Hash 表设计与冲突处理、单词分割（字符串处理）、排序算法（快速排序/归并排序）。

7. 内嵌汇编求解最大公约数（GCD）

题目：使用x86-64内联汇编（GCC扩展）实现欧几里得算法，计算两个无符号整数的GCD。

要求：

- 函数原型：unsigned int gcd_asm(unsigned int a, unsigned int b)。
- 内联汇编需使用 volatile 关键字，避免编译器优化。
- 测试用例：gcd_asm(12, 8) → 输出 4；gcd_asm(7, 5) → 输出 1。

测试输入输出：

调用 gcd_asm(12,8) → 输出 4；调用 gcd_asm(7,5) → 输出 1。

考察点：内联汇编语法（操作数约束 "r"、"a"）、寄存器操作（eax/ebx）、volatile 防止优化。

8. 检测整数无符号运算溢出

题目：实现函数检测两个无符号整数加减乘除是否溢出（超出 32 位无符号整数范围）。

要求：

- 函数原型：
  - int check_add_overflow_asm(unsigned int a, unsigned int b)
  - int check_sub_overflow_asm(unsigned int a, unsigned int b)
  - int check_mul_overflow_asm(unsigned int a, unsigned int b)
  - int check_div_overflow_asm(unsigned int a, unsigned int b)
  - 返回 1 表示溢出，0 表示正常。
- 原理：若 a + b < a（或 b），则溢出（无符号数加法溢出会回绕）。
- 测试用例：check_add_overflow(UINT_MAX, 1) → 输出 1；check_add_overflow(100, 200) → 输出 0。

测试输入输出：

输入 (UINT_MAX, 1) → 输出 1；输入 (100, 200) → 输出 0。

考察点：无符号整数溢出特性（回绕）、条件判断逻辑。

9. 32 位无符号整数字节序转换

题目：实现函数将 32 位无符号整数的字节序从大端（网络字节序）转换为小端（主机字节序），反之亦然。

要求：

- 函数原型：uint32_t swap_endian(uint32_t num)。
- 示例：输入 0x12345678（大端），输出 0x78563412（小端）。
- 测试用例：swap_endian(0x12345678) → 输出 0x78563412；swap_endian(0x78563412) → 输出 0x12345678。

测试输入输出：

输入 0x12345678 → 输出 0x78563412（用十六进制字符串表示）。

考察点：位运算（移位与掩码）、大端/小端定义、字节交换逻辑。

10. 调试宏 DEBUG_PRINT

题目：实现调试宏 DEBUG_PRINT，根据编译选项 -DDEBUG_LEVEL=1/2/3 控制输出详细程度：

- LEVEL=1：输出函数名和行号（如 DEBUG: func=main, line=10）。
- LEVEL=2：额外输出变量值（如 DEBUG: func=main, line=10, x=42）。
- LEVEL=3：输出完整调用堆栈（需调用 backtrace()）。
- **要求**：使用 __func__、__LINE__ 预定义宏。
- 测试时通过 gcc -DDEBUG_LEVEL=2 编译，调用 DEBUG_PRINT("x=%d", x)。

测试输入输出：

当 DEBUG_LEVEL=2 时，调用 DEBUG_PRINT("x=%d", 42) → 输出 DEBUG: func=test, line=20, x=42。

考察点：条件编译（#if/#elif）、预定义宏、可变参数宏（__VA_ARGS__）。

11. 简单事件处理器（回调机制）

题目：实现事件处理器，支持注册事件类型（如 EVENT_A、EVENT_B）的回调函数，并触发事件时调用对应回调。

要求：

- 事件类型用枚举定义（enum { EVENT_A, EVENT_B, EVENT_MAX }）。
- 注册函数：void register_event(enum EVENT_TYPE type, void (*callback)(void*), void* arg)。
- 触发函数：void trigger_event(enum EVENT_TYPE type)。
- 测试用例：注册 EVENT_A 的回调打印 "Event A triggered"，触发后输出该信息。

测试输入输出：

触发 EVENT_A → 输出 Event A triggered。

考察点：函数指针数组、回调机制、事件类型管理。

12. container_of 宏实现

题目：使用 GCC 扩展（如 typeof）实现 container_of(ptr, type, member) 宏，通过结构体成员的指针获取结构体变量的指针。

要求：

- 示例：定义 struct Test { int a; char b; }，若 ptr 是 &test.b，则 container_of(ptr, struct Test, b) 应返回 &test。
- 测试用例：定义结构体变量 t，取成员 t.b 的指针传入宏，验证返回的指针是否等于 &t。

测试输入输出：

结构体 t 的地址为 0x7ffd...，成员 t.b 的指针传入宏后，输出 0x7ffd...（与 &t 相同）。

考察点：指针运算、typeof 扩展、结构体成员偏移量计算。

13. 可扩展动态数组（类似 QEMU GArray）

题目：实现动态数组 GArray，支持自动扩容（初始容量 16，每次扩容 2 倍），提供初始化、追加元素、释放内存的接口。

要求：

- 结构体定义：typedef struct { void *data; size_t len; size_t capacity; } GArray。
- 接口：GArray* garray_init(size_t elem_size)、void garray_append(GArray *arr, void *elem)、void garray_free(GArray *arr)。
- 测试用例：初始化 GArray 存储 int，追加 [1,2,3,16]（超过初始容量 16？不，初始容量是元素个数，这里假设初始容量 16 个 int，追加 16 个元素后扩容到 32）。

测试输入输出：

追加 17 个 int 元素后，检查 arr->len=17，arr->capacity=32，且元素值正确。

考察点：内存管理（realloc）、泛型编程（void*）、扩容策略设计。

14. 紧凑网络协议头解析器（位域）

题目：定义网络协议头结构体（使用位域），解析二进制字节流并输出各字段值。

要求：

- 协议头格式：2 字节版本（4 位主版本+4 位次版本）、2 字节长度（16 位无符号）、1 字节标志（3 位保留+5 位功能标志）。
- 输入字节流示例：\x00\x03\x00\x20\x00（版本 0.3，长度 32，标志 0x00）。
- 输出：version:0.3, length:32, flags:0x00。

测试输入输出：

输入字节流 \x00\x03\x00\x20\x00 → 输出 version:0.3, length:32, flags:0x00。

考察点：位域定义（struct { unsigned int ver_major:4; ... }）、结构体对齐（#pragma pack(1)）、字节流解析。

15. 简易 ELF 信息查看工具

题目：解析 ELF 文件头，输出 ELF 类型（可执行/共享库）、入口地址、程序头表中加载段的内存范围。

要求：

- 读取 /usr/include/elf.h 中的 Elf64_Ehdr 结构体定义。
- 输入 ELF 文件路径（如 /bin/ls）。
- 输出示例：Type: ET_EXEC, Entry: 0x401000, Load segments: 0x400000-0x402000, 0x402000-0x403000。

测试输入输出：

输入 /bin/ls → 输出包含 ET_EXEC、入口地址、加载段范围的正确信息。

考察点：二进制文件 I/O（fread）、结构体与二进制数据对齐、ELF 格式解析。

16. LRU 缓存淘汰算法

题目：实现基于哈希表和双向链表的 LRU 缓存，支持插入（put）和查询（get）操作，容量满时淘汰最久未使用的元素。

要求：

- 结构体包含哈希表（快速查找）和双向链表（维护访问顺序）。
- 输入：容量 3，依次 put(1,1)、put(2,2)、put(3,3)、put(4,4)（淘汰 1），然后 get(2)（更新 2 为最近使用），再 put(5,5)（淘汰 3）。

- 输出：最终缓存内容为 2:2, 4:4, 5:5。

测试输入输出：

操作序列后，缓存键集合为 {2,4,5}（顺序无关）。

考察点：哈希表与双向链表的协同、LRU 淘汰策略（最近最少使用）。

17. 位图操作

题目：实现位图（Bitmap）的置位（set_bit）和查询（test_bit）功能。

要求：

- 位图用 unsigned char* 数组存储，支持按位操作。
- 输入：位图大小 10，置位第 0、3、5 位，查询第 3 位应返回 1，查询第 1 位返回 0。
- 测试用例：set_bit(bitmap, 0) → test_bit(bitmap, 0)=1；test_bit(bitmap, 1)=0。

测试输入输出：

置位 0、3、5 后，查询 3 → 输出 1；查询 1 → 输出 0。

考察点：位运算（|=、&=、(1<<n)）、数组索引计算（byte = bit / 8，offset = bit % 8）。

18. 线程安全环形缓冲区

题目：实现线程安全的环形缓冲区（Ring Buffer），支持多线程生产者和消费者模型。

要求：

- 使用互斥锁（pthread_mutex_t）和条件变量（pthread_cond_t）保证线程安全。
- 输入：缓冲区容量 5，生产者线程写入 [1,2,3,4,5,6]（第 6 个元素等待消费者读取后写入），消费者线程读取并打印。
- 输出：消费者打印 1,2,3,4,5,6（顺序正确，无数据丢失）。

测试输入输出：

生产者写入 6 个元素，消费者读取 6 个元素，输出顺序与写入顺序一致。

考察点：多线程同步（互斥锁、条件变量）、环形缓冲区的读写指针管理。

19. 字符串分割器（类似 strtok_r）

题目：实现线程安全的字符串分割函数 strtok_r，支持指定分隔符集合。

要求：

- 函数原型：char* strtok_r(char *str, const char *delim, char **saveptr)。
- 输入：字符串 "hello,world test"，分隔符 ", "（逗号和空格）。
- 输出：分割结果 ["hello", "world", "test"]。

测试输入输出：

调用 strtok_r 三次，依次输出 hello、world、test。

考察点：状态保存（saveptr 指针）、分隔符匹配、线程安全（无静态变量）。

20. 位图实现 Bloom 过滤器

题目：使用位图和多个哈希函数实现 Bloom 过滤器，支持插入元素和查询元素是否存在。

要求：

- 位图大小 m=100，哈希函数数量 k=3（如 hash1(str) =sum(c) % m，hash2(str)=sum(c*2)%m，hash3(str)=sum(c*3)%m）。
- 输入：插入 "apple"、"banana"，查询 "apple"（应存在）、"orange"（可能存在误判）。
- 输出：apple exists: 1，orange exists: 0 或 1（允许误判）。

测试输入输出：

插入后查询 apple → 输出 exists: 1；查询未插入的 grape → 输出 exists: 0（理想情况）或 1（误判）。

考察点：哈希函数设计、位图操作、Bloom 过滤器的误判特性。